Day 14: Web/HTTP Log Inspection

Identifying Client IP, Method, Path, Status Code, and User-Agent with grep

Field	Details
Intern Name	Amrita Pathak
Date	16 September 2026
Internship Track	SOC (Security Operations Center)
Task Deadline	16 September 2026, 11:59 PM IST

Table of Contents

Table of Contents 2
Key Terms 3
Executive Summary 4
1. Objective 4
2. Data and Environment Declaration 4
3. Tools and Environment 5
4. Why This Task Matters 6
5. Generating and Analyzing Web Traffic 6
5.1 Session Steps 6
5.2 What Each grep Command Does 13
6. Results Summary 13
7. Findings 14
7.1 The 404 cluster shows a believable reconnaissance pattern 14
7.2 Not every 404 in the log is suspicious 14
7.3 A faked user-agent on a valid path is a different signal than a 404 probe 14
7.4 A display artifact was caught before it became a false finding 14
8. Conclusion 14

Key Terms

- **Client IP**: the address of whoever made the request, identifying which machine on the network asked for something. In this exercise, 127.0.0.1 (loopback) and 192.168.84.128 (the VM's real network address) were used deliberately to produce two distinct sources from the same physical machine.

- **Request method**: the type of action requested. GET, used throughout this exercise, asks to retrieve something, such as a page. POST, not used here, asks to submit data, such as a login form.

- **URL / path**: the specific resource requested on the server, such as / or /admin, the part of the address after the domain or IP.

- **Status code**: the server's numeric response to a request. 200 means success. 404 means the requested resource does not exist. Codes are grouped by their first digit, 4xx for client-side problems, 5xx for server-side ones, which is what makes a single regular expression able to catch both categories at once.

- **User-agent**: a text string a client sends identifying what made the request, a real browser, a command-line tool like curl, or, in the case of an automated scanner, sometimes the tool's own name outright. This field is a common place for reconnaissance tools to reveal themselves, and also a field attackers sometimes deliberately fake to blend in with normal browser traffic.

- **Reconnaissance probing**: requests aimed at paths commonly associated with sensitive files or admin interfaces, such as /wp-login.php or /.env, made to discover what exists on a server before attempting anything further. A cluster of 404s against paths like these, rather than one isolated 404, is a recognizable scanning pattern.

Executive Summary

This task generated real web traffic against a local test server on the Kali lab VM, identified all five core fields a web log records for every request, client IP, request method, URL path, status code, and user-agent, and used grep to flag unusual status codes and repeated requests, exactly as the task specified.

Traffic was generated two distinct ways to produce genuine variety rather than a single flat pattern: normal browsing through Firefox against the loopback address, and scanning-style requests through curl against the machine's real network IP, including three probes against nonexistent paths, one request with a deliberately faked scanner user-agent, and a rapid five-request burst against the same path. A mid-session check also confirmed that an apparent missing user-agent field was a terminal display issue, not a real data problem, before any conclusions were drawn from it.

The final log contained 13 requests. Filtering by status code isolated 4 non-success responses, all 404s. Filtering by user-agent isolated the one request faking the sqlmap scanner. Counting repeated requests to the same path from the same source confirmed exactly 6, matching the traffic deliberately generated for that purpose.

1. Objective

Generate requests to a local test web service, identify the client IP, request method, URL path, status code, and user-agent for each request, and use grep to flag unusual status codes and repeated requests within the resulting log.

2. Data and Environment Declaration

The web server used in this exercise, web_server.py, is a small script adapted from Python's standard http.server library, not written from scratch. Python's default simple HTTP server does not log the User-Agent field at all, which this task specifically requires, so a short custom script was used instead to capture all five required fields into a single log file. The script itself was not authored independently; what is demonstrated in this report is the ability to read, filter, and interpret the resulting web log, which is the actual skill this task is testing, not the ability to write server code from memory. The full script and a plain explanation of each of its parts are given below, so that the reasoning behind every line is accounted for, not just the finished output.

```
from http.server import BaseHTTPRequestHandler, HTTPServer
import datetime

class LoggingHandler(BaseHTTPRequestHandler):
    def log_and_respond(self, code, body):
        ip = self.client_address[0]
        method = self.command
        path = self.path
        ua = self.headers.get('User-Agent', '-')
        ts = datetime.datetime.now().strftime('%d/%b/%Y:%H:%M:%S')
        line = f'{ip} - - [{ts}] "{method} {path} HTTP/1.1" {code} - "{ua}"\n'
        with open('access.log', 'a') as f:
            f.write(line)
        self.send_response(code)
        self.end_headers()
        self.wfile.write(body.encode())

    def do_GET(self):
        if self.path in ('/', '/index.html'):
```

```
            self.log_and_respond(200, 'Home page')
        elif self.path == '/about':
            self.log_and_respond(200, 'About page')
        else:
            self.log_and_respond(404, 'Not Found')

HTTPServer(('0.0.0.0', 8080), LoggingHandler).serve_forever()
```

- `from http.server import BaseHTTPRequestHandler, HTTPServer`: imports Python's own built-in tools for handling web requests and running a server. Nothing here is a third-party or unfamiliar library, both come standard with Python itself.
- `class LoggingHandler(BaseHTTPRequestHandler)`: defines how every incoming request should be handled, by extending Python's standard request handler with custom behavior added underneath.
- `log_and_respond(self, code, body)`: the function actually responsible for the logging this task needed. It reads the requester's IP, method, path, and user-agent directly off the incoming request, builds one line of text in a standard web-log format, and appends that line to access.log before sending the response back to whoever asked.
- `ua = self.headers.get('User-Agent', '-')`: specifically pulls the User-Agent header from the request, the exact piece missing from Python's default simple server, and the reason this script was necessary at all rather than reusing the tool from Day 7.
- `do_GET(self)`: runs automatically whenever a GET request arrives. It checks the requested path: / or /index.html returns a 200 with a home page message, /about returns a 200 with an about page message, and anything else at all falls through to the else branch and returns a 404, which is exactly why /admin, /wp-login.php, and /.env all came back as 404s, none of those paths were ever defined as valid.
- `HTTPServer(('0.0.0.0', 8080), LoggingHandler).serve_forever()`: starts the server itself, listening on port 8080 across all of the machine's network interfaces (0.0.0.0), which is what allowed it to accept requests aimed at both 127.0.0.1 and the VM's real address, 192.168.84.128, and keeps running indefinitely until manually stopped.

All traffic in this exercise was generated deliberately, by the same person operating the same machine, for the purpose of this exercise. The scanning-style requests, the faked sqlmap user-agent, and the repeated-request burst are simulated patterns built to resemble real reconnaissance behavior, not activity directed at any system outside this local lab environment.

3. Tools and Environment

Item	Detail
Endpoint used	Kali Linux VM (kali)
Test service	Custom Python HTTP server (web_server.py), logging to access.log
Traffic sources	Firefox (127.0.0.1), curl (192.168.84.128)
Tools used	browser, curl, grep

Item	Detail
Log size	13 requests total

4. Why This Task Matters

Every log format examined so far in this internship recorded authentication activity. Web logs record something different: who requested what, and what the server said back, and a large share of real attacks against web-facing systems show up first as patterns in exactly this kind of log, long before any authentication is ever attempted. The specific value this task builds is worth stating directly:

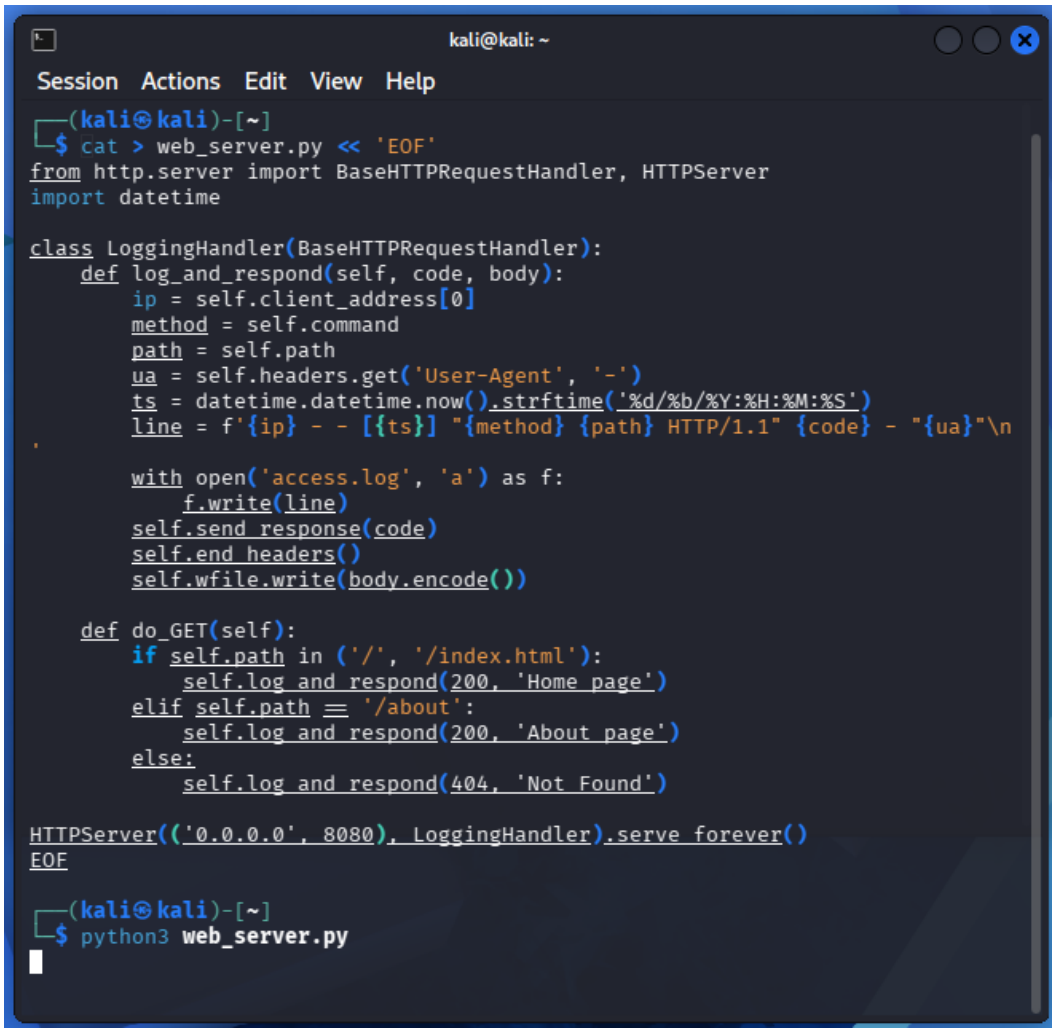
- **Reconnaissance is visible before compromise**: a scanner probing for /admin, /wp-login.php, or /.env leaves a trail of 404s well before it ever finds something real to exploit. An analyst who can recognize this pattern early sees an attack in its reconnaissance stage, not only after something has already gone wrong.
- **Status codes are a fast triage signal**: grouping by the first digit of a status code, 4xx versus 5xx versus 2xx, gives an analyst an immediate sense of what a burst of traffic represents without reading every line individually, exactly the same triage instinct built in earlier log-hunting exercises, applied to a new log format.
- **The user-agent field cuts both ways**: it can expose a scanner that announces itself outright, and it can just as easily be faked by something less obvious that copies a real browser's string. Neither possibility should be assumed by default; the field has to actually be read, not just glanced at.
- **Verifying before concluding still applies here**: a formatting issue during this exercise briefly looked like a missing user-agent field. Checking a single line in isolation before treating that as a real finding avoided reporting a data problem that was never actually there. The same discipline applied throughout this internship, now exercised on a new kind of evidence.

5. Generating and Analyzing Web Traffic

This section documents the full working session, from building the test server through to a graceful shutdown, followed by an explanation of what each grep command used, actually does.

5.1 Session Steps

- **Step 1 - created and started the logging server:** wrote `web_server.py` to disk using a heredoc, a script that responds to requests for / and /about with a 200, anything else with a 404, and logs the client IP, method, path, status code, and user-agent for every single request to `access.log`. Started it with `python3 web_server.py` and left it running in its own terminal for the rest of the session.

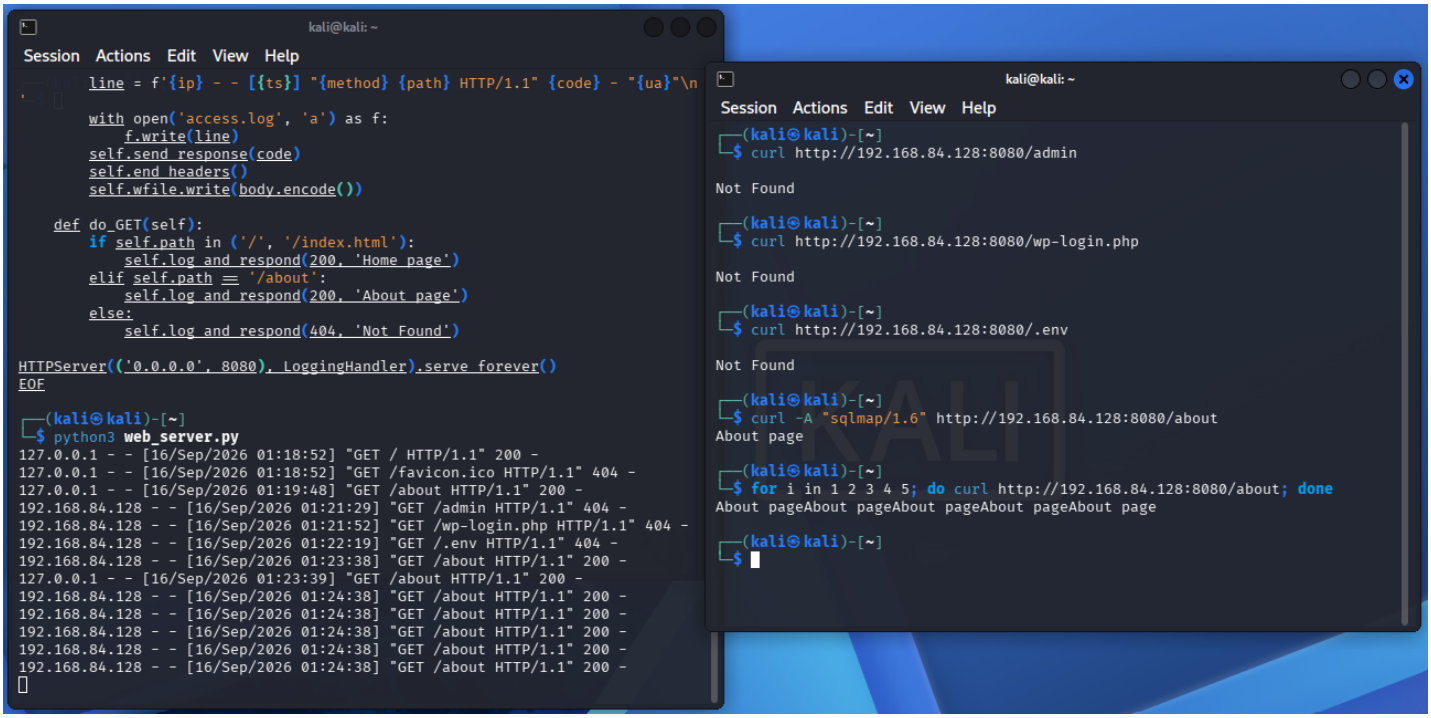


The logging server script and its startup, left running to capture every request made during this exercise.

- **Step 2 - generated normal browsing traffic:** visited `http://127.0.0.1:8080/` and `http://127.0.0.1:8080/about` directly in Firefox, producing two legitimate GET requests with Firefox's real user-agent string, using the loopback address to keep this traffic source distinct from the scanning-style traffic that followed.

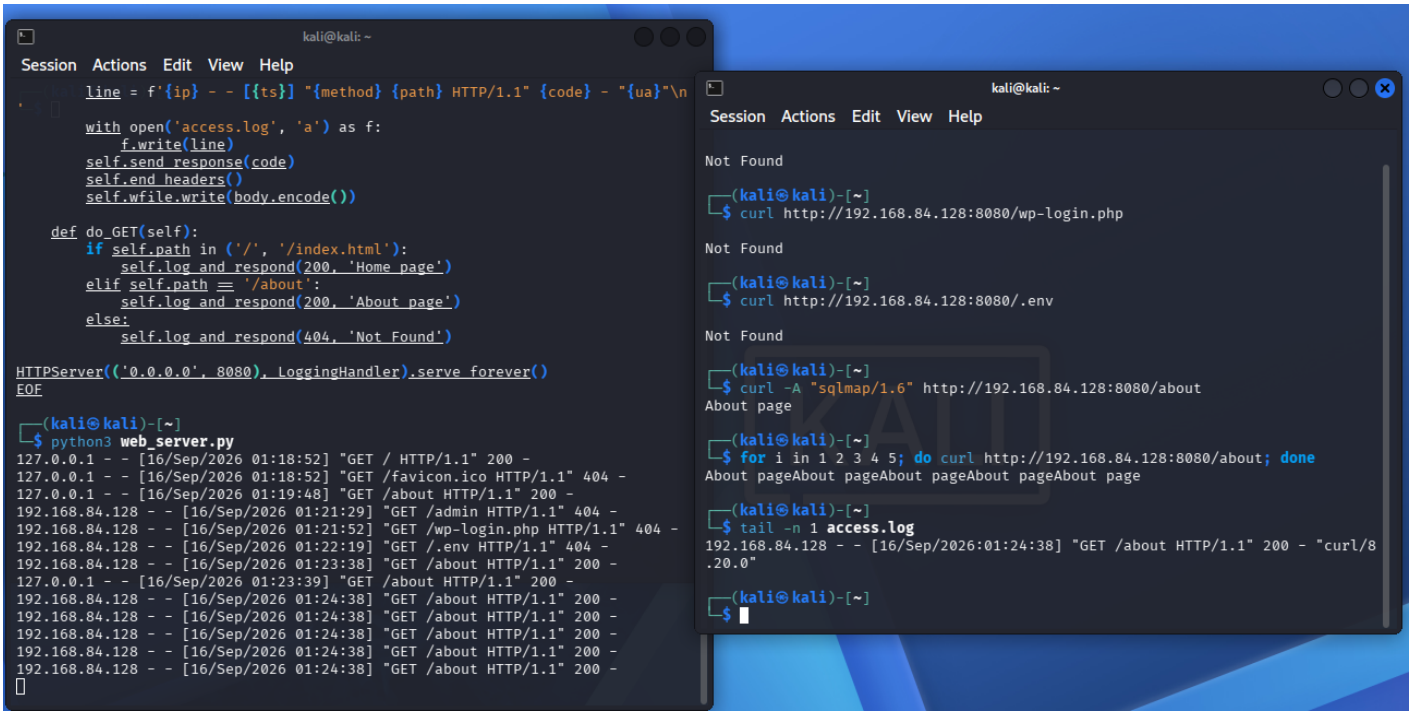
- **Step 3 - generated scanning-style traffic with curl:** ran four separate curl requests against the VM's real network address, 192.168.84.128: three targeting paths that do not exist on this server (`/admin`, `/wp-login.php`, `/.env`), each a real path automated scanners commonly probe for, and one request to the valid `/about` path using curl's `-A` flag to fake the user-agent as `"sqlmap/1.6"`, sqlmap being a real, well-known automated SQL injection tool.

- **Step 4 - generated a repeated-request burst:** ran `for i in 1 2 3 4 5; do curl http://192.168.84.128:8080/about; done`, firing five identical requests to the same path in immediate succession, simulating the kind of repetitive, scripted behavior the task specifically asks to be flagged.



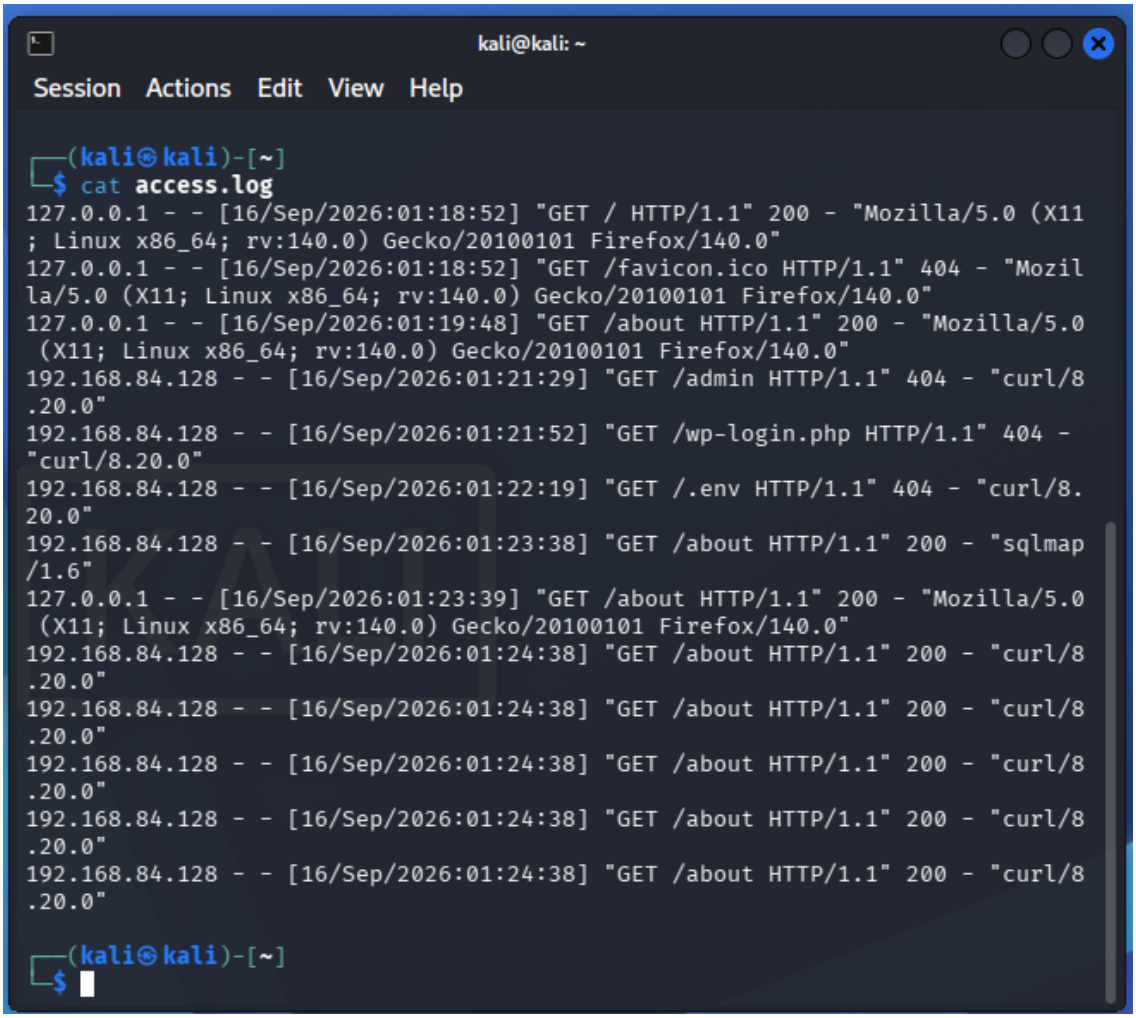
The scanning-style curl requests and the repeated-request loop, alongside the server's terminal showing entries already being written to the log.

- **Step 5 - verified a formatting concern before trusting the data:** a combined view of two terminal windows side by side made every log line appear to end abruptly after the status code, with no visible user-agent field at all. Rather than accepting that as a real finding, `tail -n 1 access.log` was run in a single, full-width terminal to check one line in isolation.



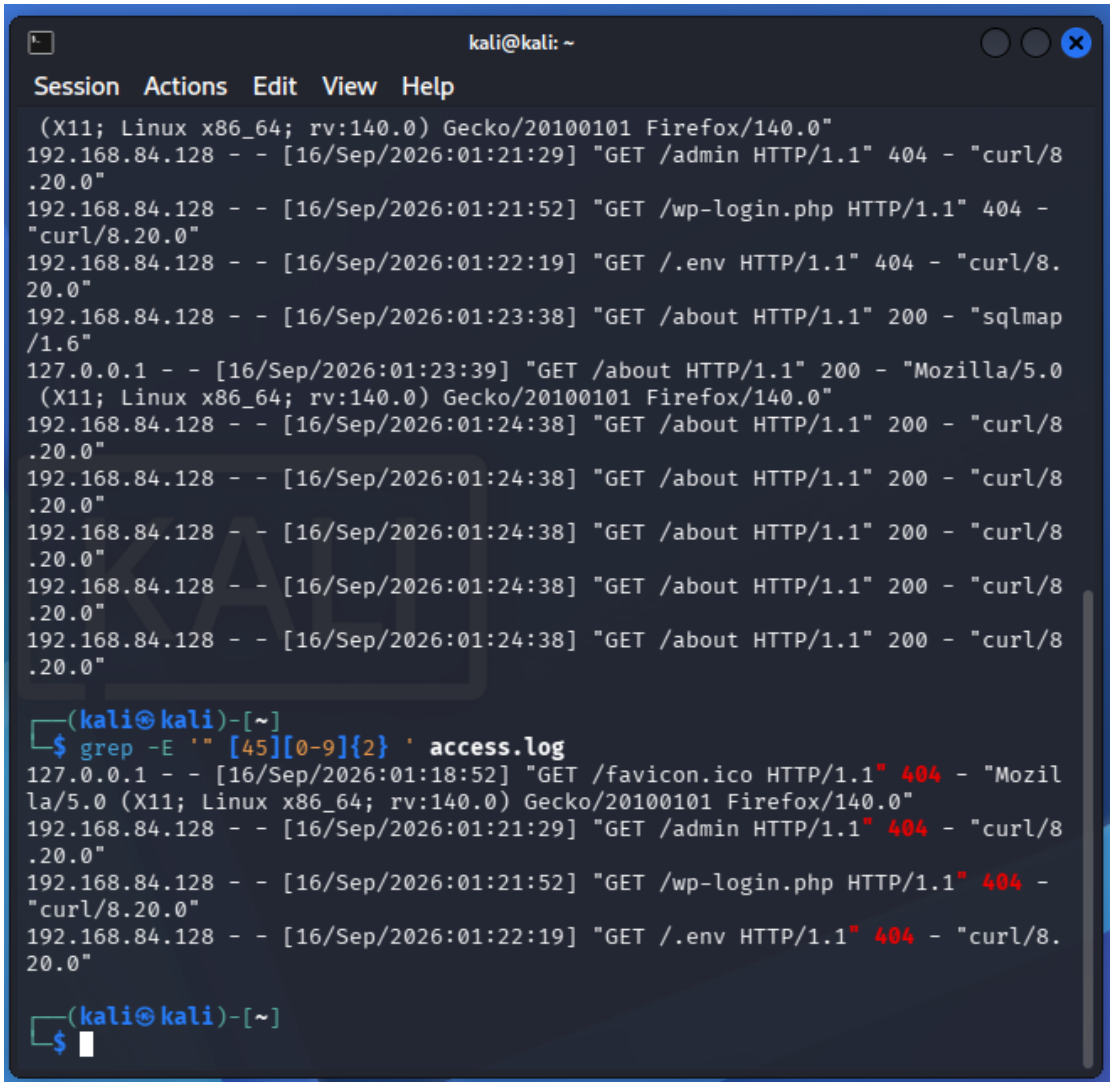
A single log line viewed in isolation, confirming the user-agent field (curl/8.20.0) was present all along, the earlier appearance of a missing field was caused by two narrow terminal windows cutting off long lines, not a real gap in the data.

- **Step 6 - reviewed the complete log:** ran `cat access.log` in a single full-width terminal, confirming all 13 requests were captured correctly with every field intact, before running any filtering commands against it.



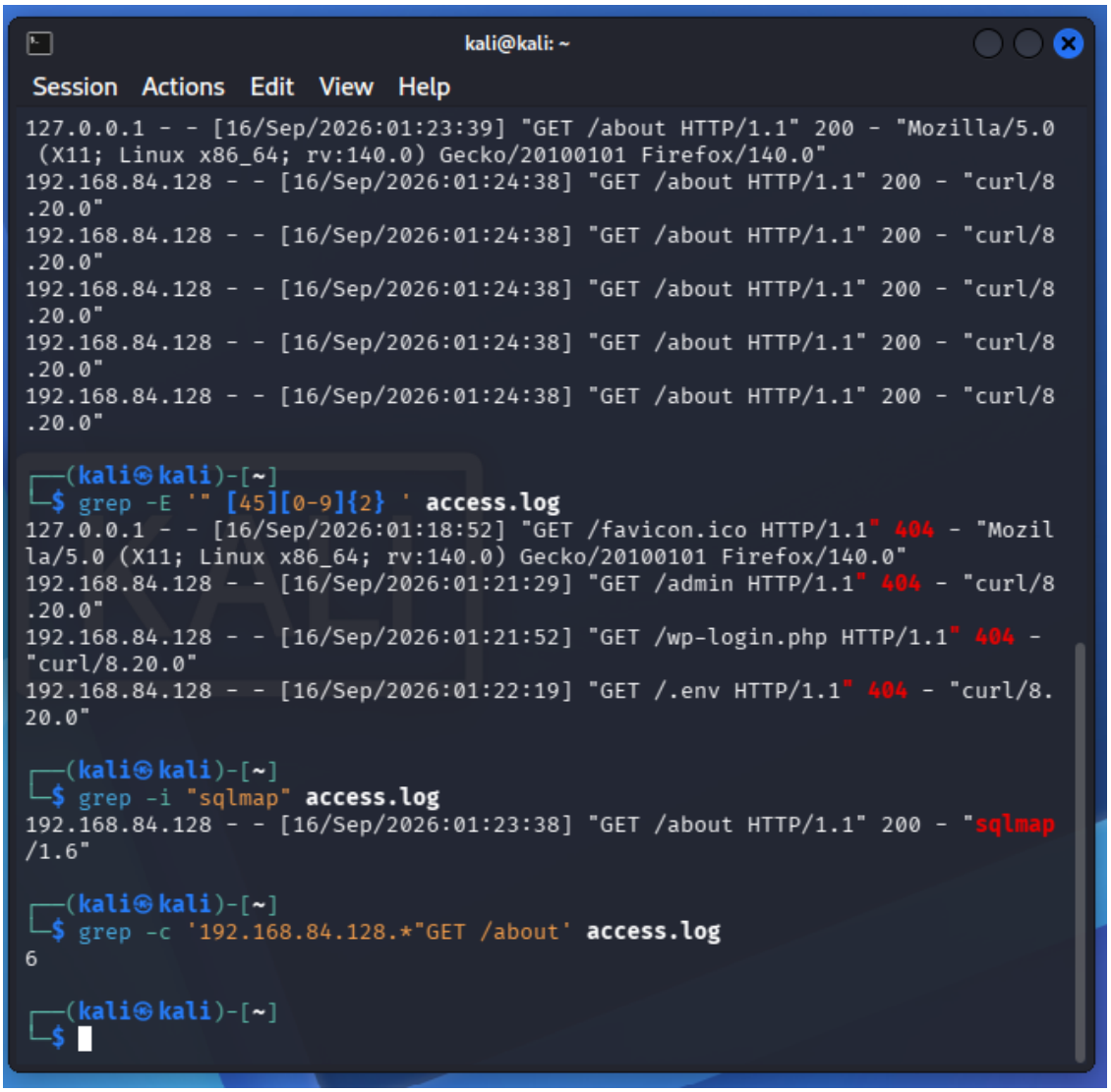
The complete 13-line access.log, all five required fields visible and intact for every request.

- **Step 7 - flagged unusual status codes:** ran `grep -E '" [45][0-9]{2} ' access.log`. Result: 4 matching lines, all 404s, covering /favicon.ico, /admin, /wp-login.php, and /.env.



grep isolating all 4xx and 5xx status codes: four 404 responses.

- **Step 8 - flagged the suspicious user-agent and quantified the repeated requests:** ran `grep -i "sqlmap" access.log`, isolating the one request faking that user-agent, then ran `grep -c '192.168.84.128.*"GET /about' access.log`. Result: 6, matching the five looped curl requests plus the one sqlmap request, both hitting the same path from the same source.



The sqlmap user-agent isolated, and the repeated-request count confirmed as 6.

- **Step 9 - stopped the server:** switched to the server's terminal and pressed `Ctrl+C`, cleanly interrupting the running script. Since this was a manually run process rather than a background service, no separate stop command was needed beyond ending it directly.

- **Step 10 - shut down the Kali VM gracefully:** ran `sudo shutdown now` rather than force-closing the VM window, letting the filesystem unmount cleanly, the same habit followed after every Splunk session earlier in this internship.

5.2 What Each grep Command Does

- `grep -E '" [45][0-9]{2} ' access.log`: the -E flag enables extended regular expressions, needed here for the {2} repetition syntax. The pattern looks for a quote mark and a space, then a digit that is either 4 or 5, then exactly two more digits, then another space, matching any 4xx or 5xx status code surrounded by the quote marks and spaces that appear around it in this log's format, without accidentally matching a 4 or 5 appearing elsewhere in the line, such as inside a timestamp or an IP address.

- `grep -i "sqlmap"`: -i makes the search case-insensitive, so SQLMap, sqlMap, or sqlmap would all match. The plain string search looks for that word anywhere in a line, no regular expression needed since a literal name has no ambiguity to resolve.

- `grep -c '192.168.84.128.*"GET /about' access.log`: -c tells grep to print a count of matching lines instead of the lines themselves. The pattern combines two conditions into one, this specific source IP, followed by anything (.*), followed by a GET request to /about, only counting lines where both conditions are true on the same line.

6. Results Summary

Item	Value
Total requests logged	13
Distinct client IPs	2 (127.0.0.1 legitimate; 192.168.84.128 scanning-style)
Non-success status codes (4xx/5xx)	4, all 404s
Suspicious user-agent detected	1 request faking sqlmap/1.6
Repeated requests (same IP, same path)	6, to /about from 192.168.84.128
Data integrity check performed	Confirmed apparent missing user-agent was a display artifact, not real

7. Findings

7.1 The 404 cluster shows a believable reconnaissance pattern

Three of the four 404s, /admin, /wp-login.php, and /.env, are all real paths automated scanners commonly probe for on any web server. Seeing three such probes from the same source in quick succession, rather than one isolated 404, is what actually makes a pattern worth flagging, a single 404 alone proves very little.

7.2 Not every 404 in the log is suspicious

The fourth 404, /favicon.ico, came from Firefox automatically, requesting a tab icon in the background during normal browsing, not from anything deliberately probed. Grouping all four 404s together without reading each one individually would have overstated how much of this log actually reflects scanning behavior.

7.3 A faked user-agent on a valid path is a different signal than a 404 probe

The sqlmap-flagged request hit /about, a page that genuinely exists, and received a 200, not a 404. This is a meaningfully different pattern from the reconnaissance cluster: rather than searching for something that might exist, this represents a tool interacting with a known, valid resource while misrepresenting what it is, exactly the kind of detail that is only visible by actually reading the user-agent field, not just the status code.

7.4 A display artifact was caught before it became a false finding

Viewing two narrow terminal windows side by side made every log line appear to be missing its user-agent field entirely. Checking a single line in isolation with tail before treating that as a real gap in the data prevented a formatting issue from being reported as a genuine finding about the logging script itself.

8. Conclusion

A local test web server was built specifically to capture all five fields this task required: client IP, request method, URL path, status code, and user-agent, since the more common default server does not log the last of these at all. Deliberately varied traffic, legitimate browsing, reconnaissance-style probing, a faked scanner identity, and a repeated-request burst, were generated across two distinct source addresses on the same machine, then filtered using only grep, exactly as the task specified.

The most useful result here was not any single flagged line, but the discipline applied around them: separating an explainable 404 from a suspicious one, recognizing that a faked user-agent on a valid page tells a different story than a probe against a missing one, and catching a display artifact before it was mistaken for a real gap in the evidence. That same discipline, verify before concluding, is what has carried through every log format examined in this internship so far.